Sequence Diagram Descriptions

1. Add Paper to Corpus (Admin)

Overview

This sequence diagram shows how an Admin adds a new research paper to the system's corpus. The paper is fetched from an external source, parsed, tagged with rhetorical moves, and stored so it can later be used as an example in critique generation.

Actors and Components

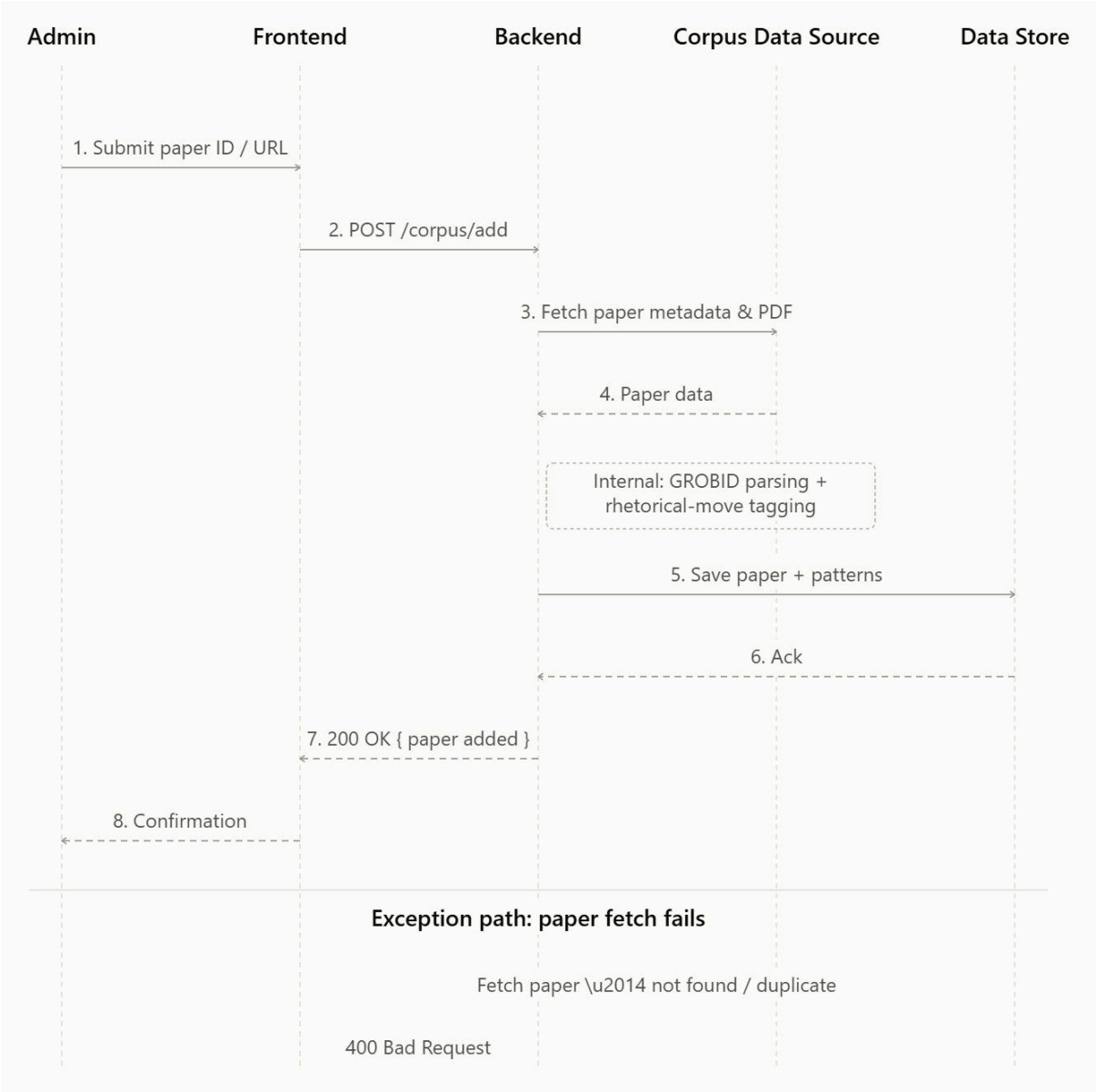
- **Admin:** The user who manages the corpus.
- **Frontend:** The web interface the Admin interacts with.
- **Backend:** The server that handles business logic, parsing, and tagging.
- **Corpus Data Source:** The external source from which paper metadata and PDFs are retrieved.
- **Data Store:** The database where papers and extracted patterns are saved.

Main Flow

1. The Admin submits a paper ID or URL through the Frontend.
2. The Frontend sends a POST request to the Backend at `/corpus/add`.
3. The Backend requests the paper metadata and PDF from the Corpus Data Source.
4. The Corpus Data Source returns the paper data to the Backend.
5. The Backend internally parses the PDF using GROBID and tags the text with rhetorical moves.
6. The Backend saves the paper and the extracted patterns to the Data Store.
7. The Data Store sends an acknowledgment back to the Backend.
8. The Backend returns a 200 OK response (paper added) to the Frontend.
9. The Frontend displays a confirmation message to the Admin.

Exception Path: Paper Fetch Fails

If the Corpus Data Source cannot find the paper, or the paper already exists in the corpus (duplicate), the fetch fails. The Backend then returns a 400 Bad Request response to the Frontend, and the Admin is informed that the paper could not be added.



2. Generate Critique (Student)

Overview

This sequence diagram shows how a Student receives feedback on a draft. The Backend sends the draft to an LLM along with a prompt and example patterns, receives a critique, saves it to the Student's history, and returns it to be displayed.

Actors and Components

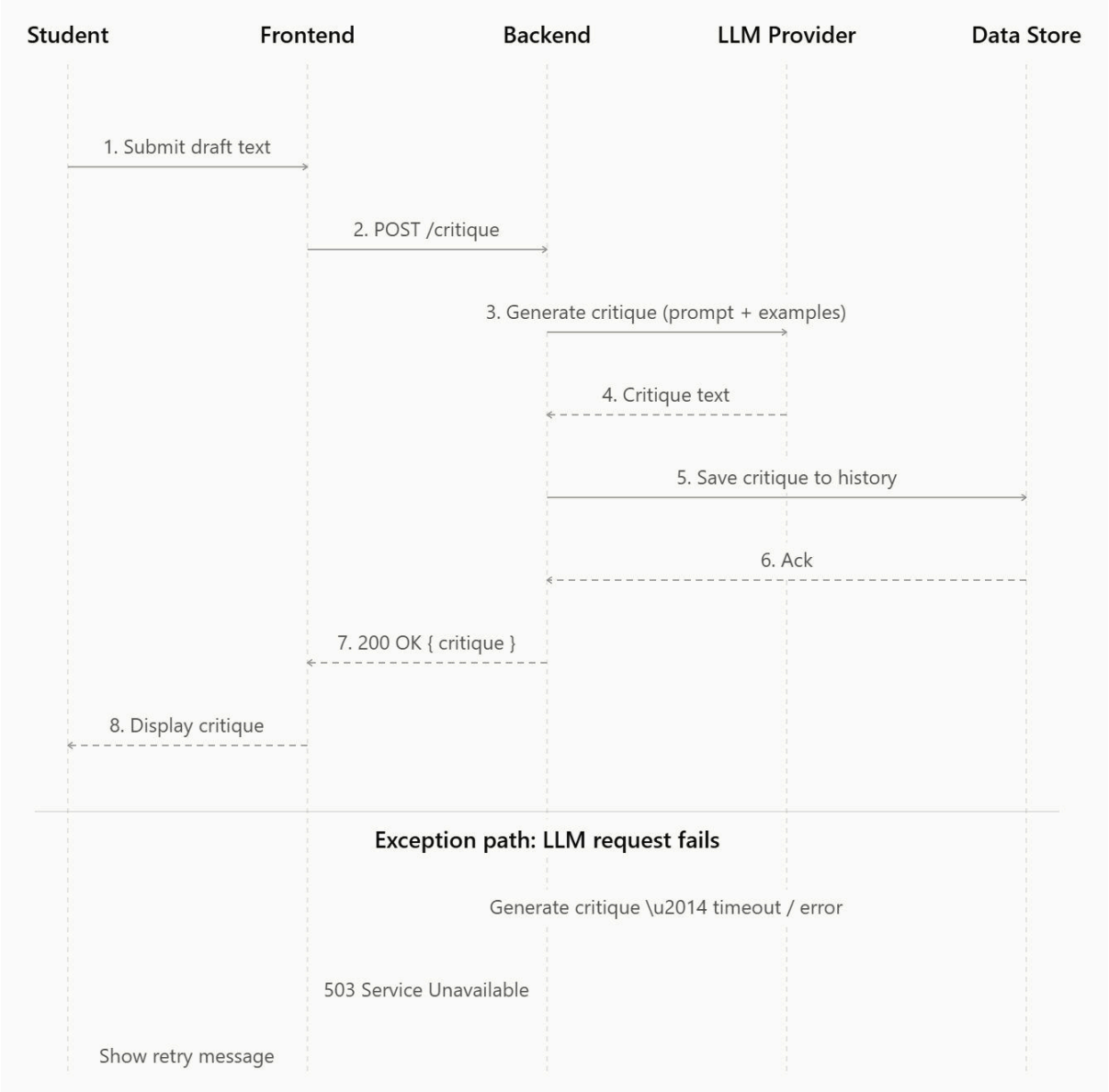
- **Student:** The user who submits draft text for feedback.
- **Frontend:** The web interface the Student interacts with.
- **Backend:** The server that builds the prompt, communicates with the LLM, and manages history.
- **LLM Provider:** The external large language model service that generates the critique.
- **Data Store:** The database where critique history is stored.

Main Flow

1. The Student submits draft text through the Frontend.
2. The Frontend sends a POST request to the Backend at `/critique`.
3. The Backend sends a request to the LLM Provider to generate a critique, using a prompt along with relevant examples.
4. The LLM Provider returns the critique text to the Backend.
5. The Backend saves the critique to the Student's history in the Data Store.
6. The Data Store sends an acknowledgment to the Backend.
7. The Backend returns a 200 OK response containing the critique to the Frontend.
8. The Frontend displays the critique to the Student.

Exception Path: LLM Request Fails

If the request to the LLM Provider times out or returns an error, the critique cannot be generated. The Backend returns a 503 Service Unavailable response to the Frontend, which shows the Student a retry message so they can submit again.



3. User Login (Student / Admin)

Overview

This sequence diagram shows the authentication process for both Students and Admins. The user's credentials are verified against the Data Store, and on success a session token is issued and the user is taken to their dashboard.

Actors and Components

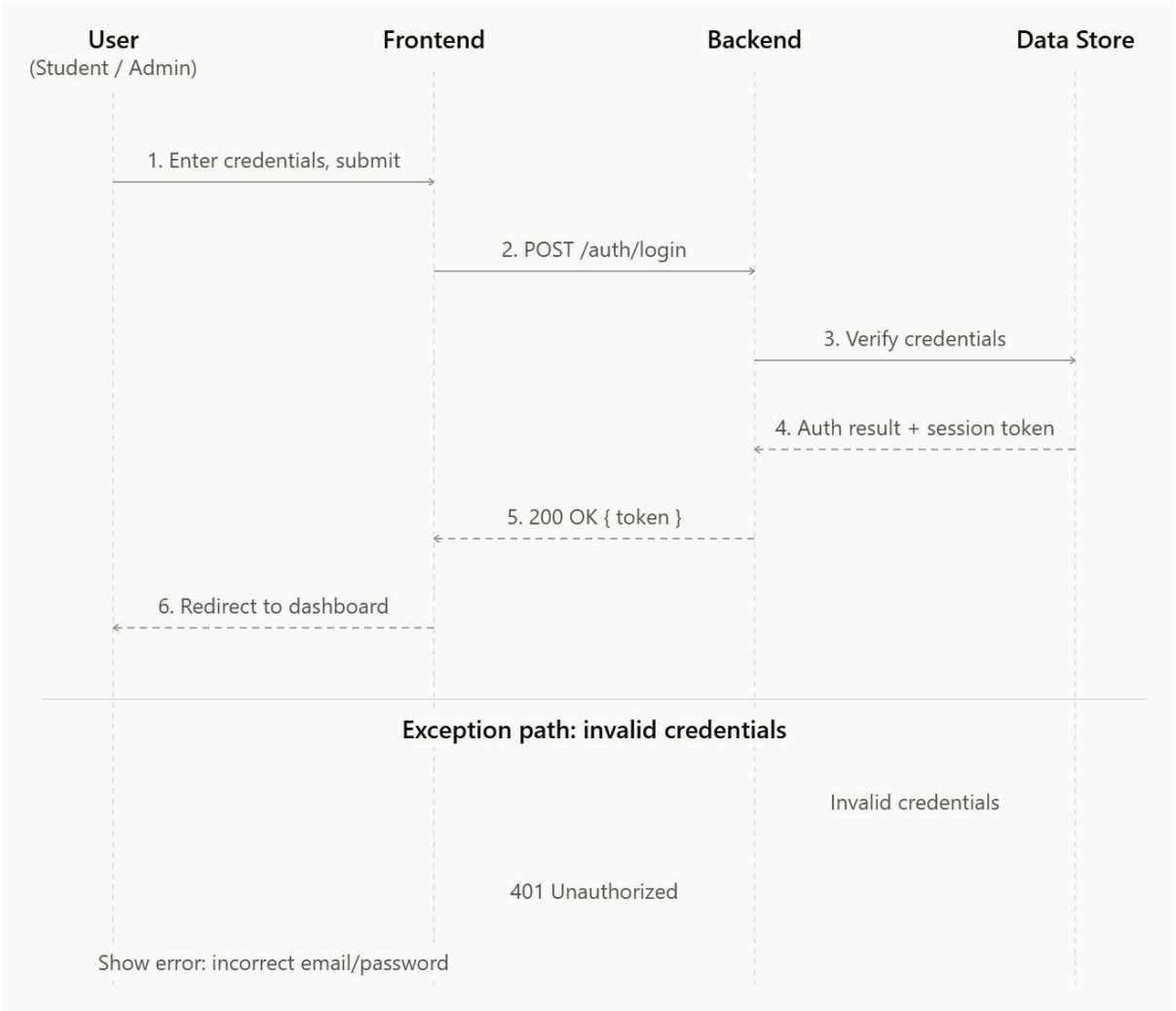
- **User (Student / Admin):** Anyone logging into the system.
- **Frontend:** The web interface containing the login form.
- **Backend:** The server that handles authentication logic.
- **Data Store:** The database holding user credentials and session information.

Main Flow

1. The User enters their credentials and submits the login form.
2. The Frontend sends a POST request to the Backend at `/auth/login`.
3. The Backend asks the Data Store to verify the credentials.
4. The Data Store returns the authentication result along with a session token.
5. The Backend returns a 200 OK response containing the token to the Frontend.
6. The Frontend redirects the User to their dashboard.

Exception Path: Invalid Credentials

If the credentials do not match the records in the Data Store, authentication fails. The Backend returns a 401 Unauthorized response to the Frontend, which displays the error message "incorrect email/password" to the User, who can then try again.


User
(Student / Admin)
Frontend
Backend
Data Store
1. Enter credentials, submit
2. POST /auth/login
3. Verify credentials
4. Auth result + session token
5. 200 OK { token }
6. Redirect to dashboard
Exception path: invalid credentials
Invalid credentials
401 Unauthorized
Show error: incorrect email/password